Encrypted Messaging and Ratcheting

CS 171: Cryptography

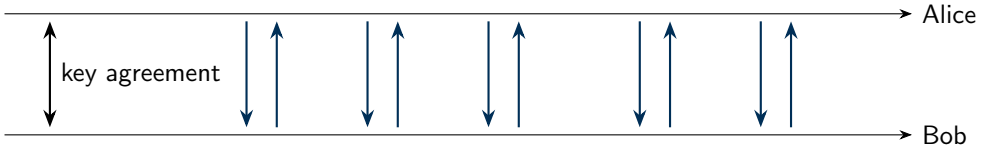
Diogo Antunes

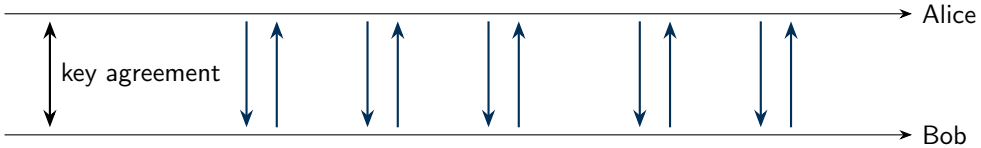
November 5, 2026

Overview

1. Forward Secrecy
2. Symmetric-Key Ratchet
3. Post-Compromise Security
4. Asymmetric-Key Ratchet
5. Double Ratchet

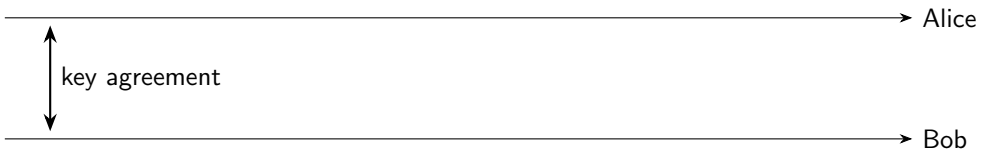

Alice
key agreement
Bob


Alice
key agreement
Bob

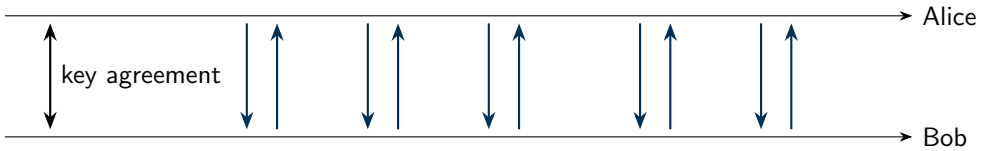


- Key agreement can be used to establish a shared secret, which is needed for symmetric cryptography.
- We saw how to achieve confidentiality and integrity with symmetric cryptography.
- This allows Alice and Bob to exchange authenticated encrypted messages.

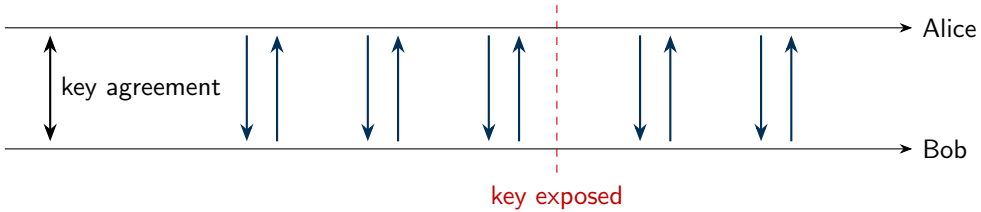
What Happens When the Key Leaks?



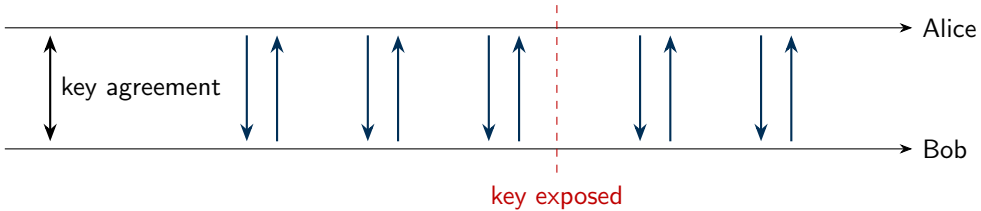
What Happens When the Key Leaks?



What Happens When the Key Leaks?

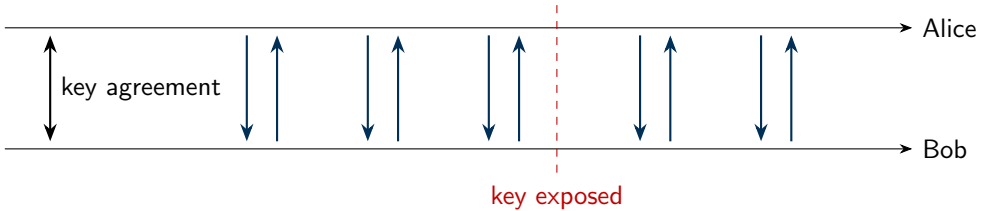


What Happens When the Key Leaks?



- So far, our security guarantees assume the key stays secret.
- Once it leaks, some exposure is unavoidable.

What Happens When the Key Leaks?

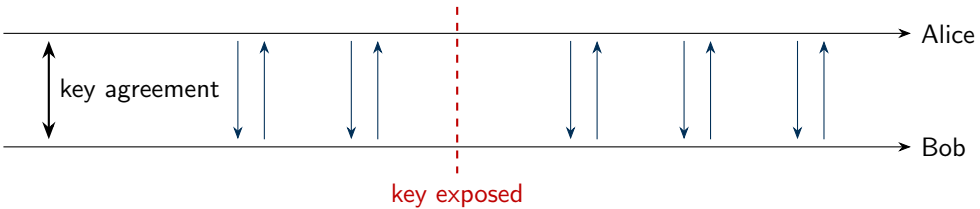


- So far, our security guarantees assume the key stays secret.
- Once it leaks, some exposure is unavoidable.

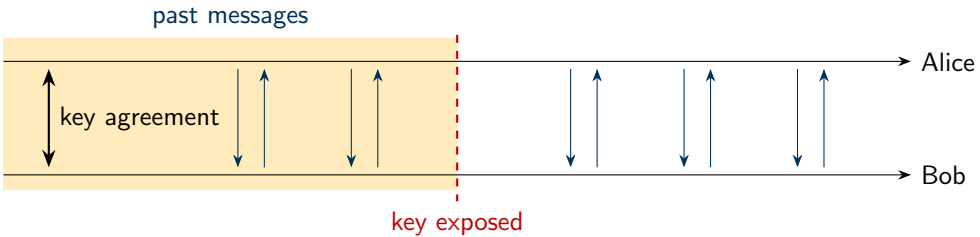
Today's question

Can we minimize the impact of a key being exposed?

Past Messages and Future Messages



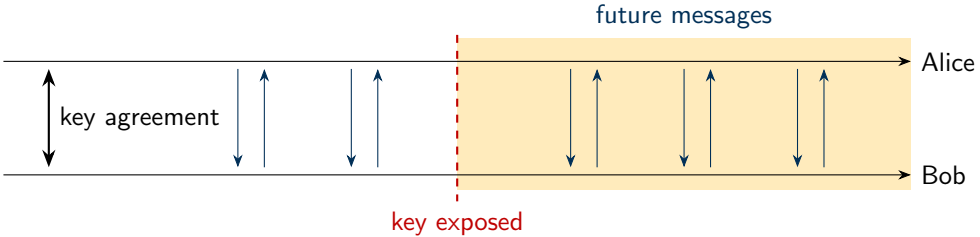
Past Messages and Future Messages



Forward secrecy

Protect past messages after current key material is exposed.

Past Messages and Future Messages



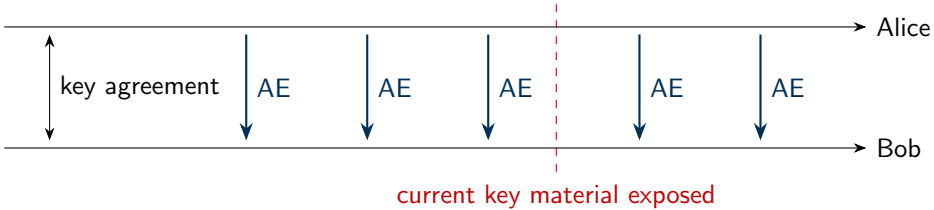
Forward secrecy

Protect past messages after current key material is exposed.

Post-compromise security

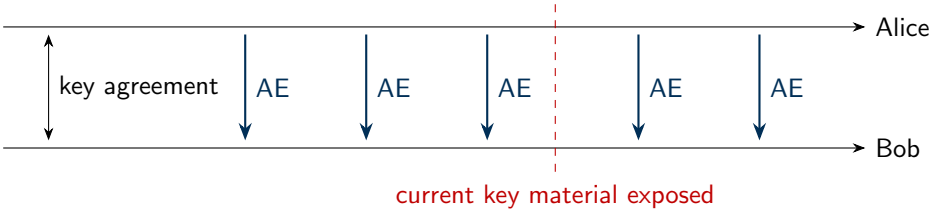
Recover secrecy for future messages after a compromise ends.

A Simpler Setting: Alice Sends to Bob

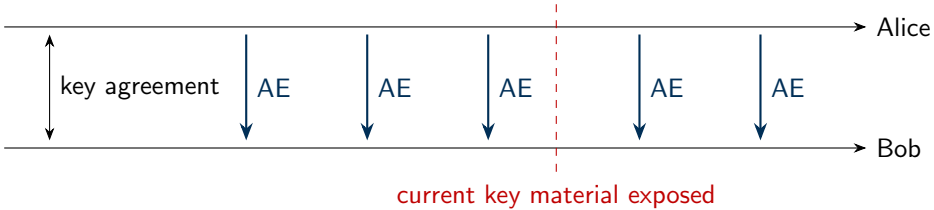


Goal: protect messages sent *before* the compromise.

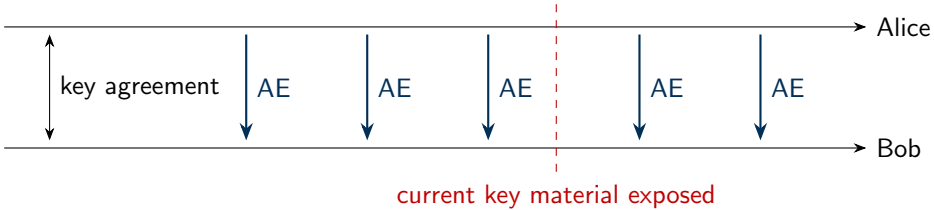
A Few Observations



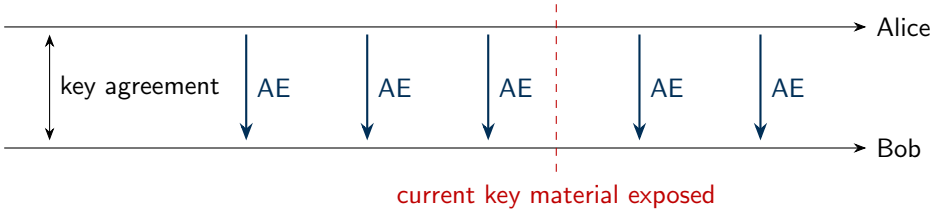
A Few Observations



1. The key must change over time.



1. The key must change over time.
2. Both parties must derive the next key from their current state.



1. The key must change over time.
2. Both parties must derive the next key from their current state.
3. Already-used keys and obsolete states must be erased.

Definition (Symmetric ratchet)

A symmetric ratchet is a pair of polynomial-time algorithms $\Pi = (\mathrm{Init}, \mathrm{Advance})$:

- $\mathrm{Init}(1^\ell)$ outputs an initial secret state st_0.

Definition (Symmetric ratchet)

A symmetric ratchet is a pair of polynomial-time algorithms $\Pi = (\mathrm{Init}, \mathrm{Advance})$:

- $\mathrm{Init}(1^\ell)$ outputs an initial secret state st_0.
- $\mathrm{Advance}(\mathsf{st})$ outputs a next state st' and a key $k \in \{0,1\}^\ell$.

Here Init is randomized; $\mathrm{Advance}$ is deterministic.

Definition (Symmetric ratchet)

A symmetric ratchet is a pair of polynomial-time algorithms $\Pi = (\mathrm{Init}, \mathrm{Advance})$:

- $\mathrm{Init}(1^\ell)$ outputs an initial secret state st_0.
- $\mathrm{Advance}(\mathsf{st})$ outputs a next state st' and a key $k \in \{0,1\}^\ell$.

Here Init is randomized; $\mathrm{Advance}$ is deterministic.

Correctness

Starting from the same state, Alice and Bob obtain the same keys and next states when they advance in sync.

Definition (Symmetric ratchet)

A symmetric ratchet is a pair of polynomial-time algorithms $\Pi = (\mathrm{Init}, \mathrm{Advance})$:

- $\mathrm{Init}(1^\ell)$ outputs an initial secret state st_0.
- $\mathrm{Advance}(\mathsf{st})$ outputs a next state st' and a key $k \in \{0,1\}^\ell$.

Here Init is randomized; $\mathrm{Advance}$ is deterministic.

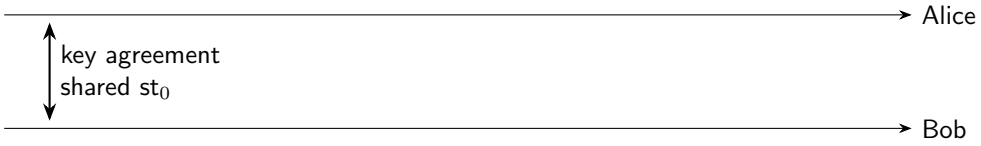
Correctness

Starting from the same state, Alice and Bob obtain the same keys and next states when they advance in sync.

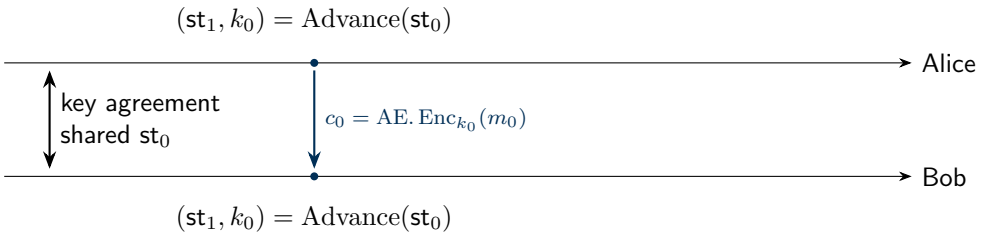
Security

Forward secrecy: past keys remain pseudorandom even when the current state is exposed.

Example: Using a Ratchet for Authenticated Encryption



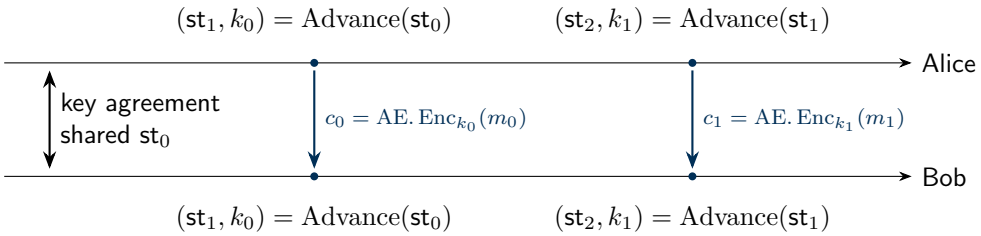
Example: Using a Ratchet for Authenticated Encryption



The ratchet key is the authenticated-encryption key

Alice encrypts with k_i; Bob derives the same k_i and computes $m_i = \mathrm{AE.Dec}_{k_i}(c_i)$.

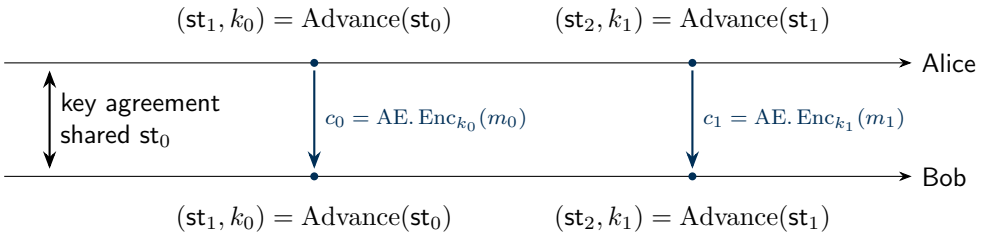
Example: Using a Ratchet for Authenticated Encryption



The ratchet key is the authenticated-encryption key

Alice encrypts with k_i; Bob derives the same k_i and computes $m_i = \mathrm{AE.Dec}_{k_i}(c_i)$.

Example: Using a Ratchet for Authenticated Encryption



The ratchet key is the authenticated-encryption key

Alice encrypts with k_i; Bob derives the same k_i and computes $m_i = \mathrm{AE.Dec}_{k_i}(c_i)$.

Each party needs to keep only the most recent state and remove the old one.

Plan of Attack

1. Build a construction and explain the intuition.

Plan of Attack

1. Build a construction and explain the intuition.
2. State the security definition.

Plan of Attack

1. Build a construction and explain the intuition.
2. State the security definition.
3. Prove security.

Question

What primitive lets us expand a short secret into a new state and a key?

Let $G : \{0,1\}^\ell \to \{0,1\}^{2\ell}$ be a secure PRG.

Construction

$\mathrm{Init}(1^\ell)$:

$\mathrm{Advance}(\mathsf{st}_i)$:

A Ratchet from a PRG

Let $G : \{0,1\}^\ell \to \{0,1\}^{2\ell}$ be a secure PRG.

Construction

$\mathrm{Init}(1^\ell)$:

Sample $\mathsf{st}_0 \leftarrow \{0,1\}^\ell$; return st_0.

$\mathrm{Advance}(\mathsf{st}_i)$:

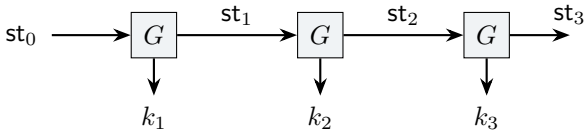
Let $G : \{0,1\}^\ell \to \{0,1\}^{2\ell}$ be a secure PRG.

Construction

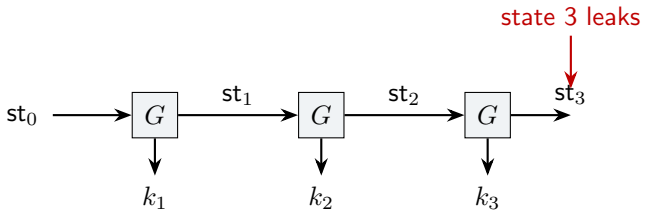
$\mathrm{Init}(1^\ell)$:
Sample $\mathsf{st}_0 \leftarrow \{0,1\}^\ell$; return st_0.

$\mathrm{Advance}(\mathsf{st}_i)$:
Parse $G(\mathsf{st}_i) = \mathsf{st}_{i+1} \| k_i$.
Return (st_{i+1}, k_i).

Intuition: Past Keys and Future Keys

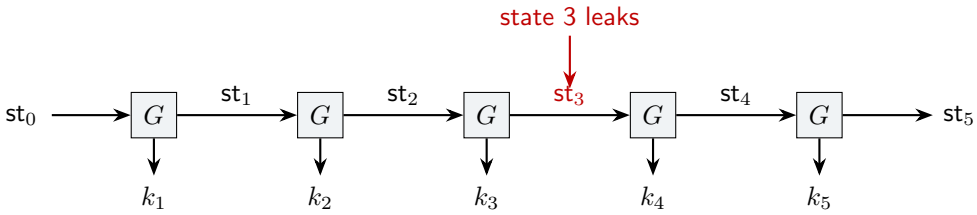


Intuition: Past Keys and Future Keys

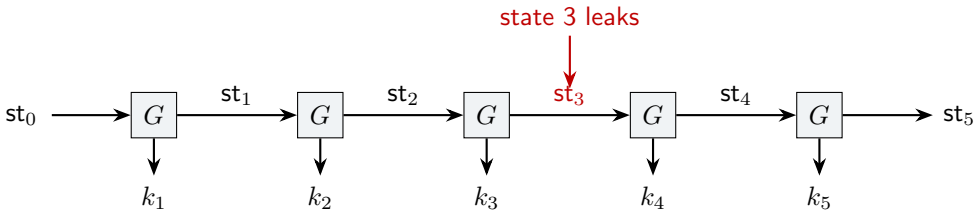


- The attacker learns st_3; earlier states and keys were erased.

Intuition: Past Keys and Future Keys



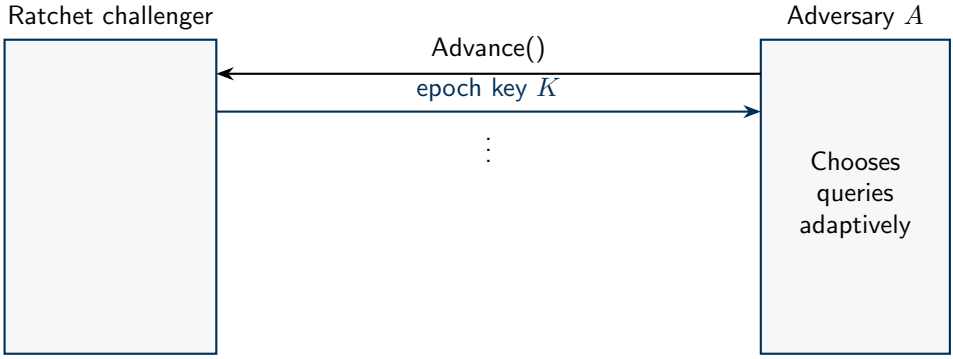
- The attacker learns st_3; earlier states and keys were erased.
- It can compute future keys.

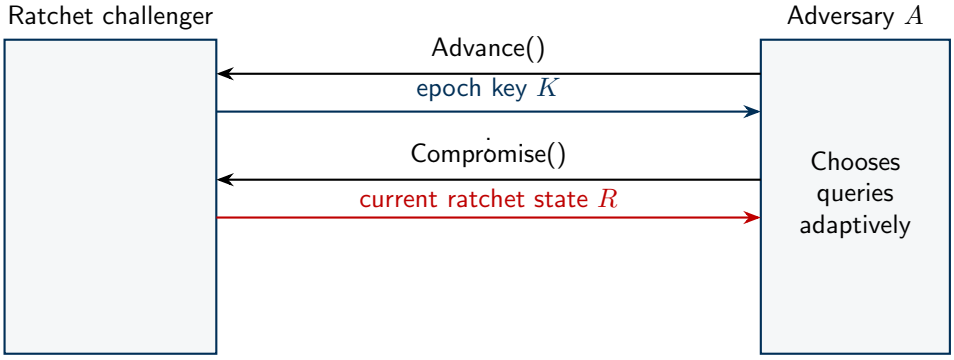


- The attacker learns st_3; earlier states and keys were erased.
- It can compute future keys.
- Forward secrecy must still protect the past keys k_1, k_2, k_3.

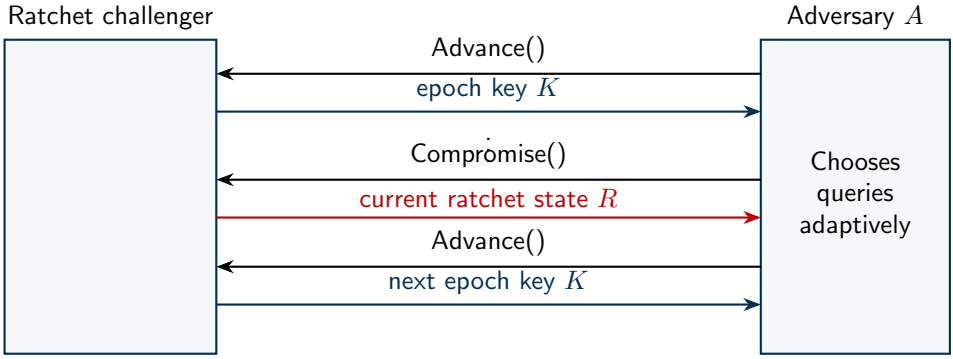
Security Game




Ratchet challenger
Adversary A
Advance()
epoch key K
Chooses queries adaptively

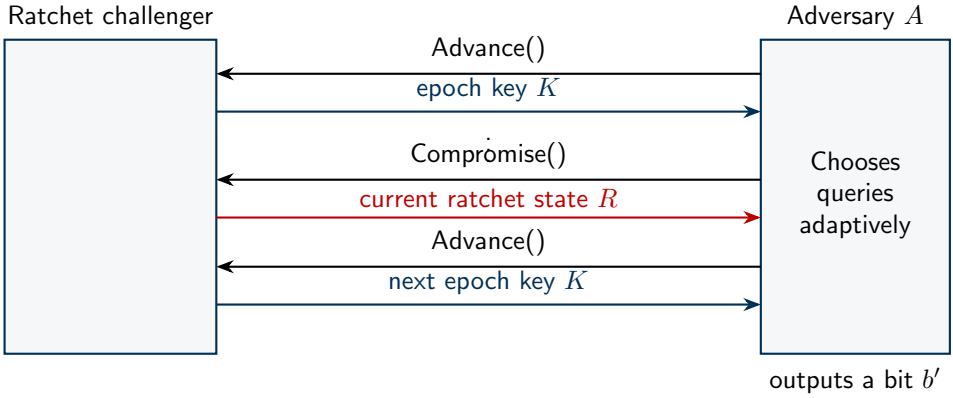


Compromise is chosen by A; further queries remain allowed.



Compromise is chosen by A; further queries remain allowed.

Security Game



Compromise is chosen by A; further queries remain allowed.

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Forward Secrecy: Two Interactive Challengers

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Ideal Ratchet

$c :=$ false; $R := \perp$.

Advance():

if not c: $K \leftarrow \mathsf{U}_\ell$.

else: $R \| K := G(R)$.

return K.

Compromise():

if not c: $R \leftarrow \mathsf{U}_\ell$.

$c :=$ true; return R.

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Ideal Ratchet

$c :=$ false; $R := \bot$.

Advance():

if not c: $K \leftarrow \mathsf{U}_\ell$.

else: $R \| K := G(R)$.

return K.

Compromise():

if not c: $R \leftarrow \mathsf{U}_\ell$.

$c :=$ true; return R.

Definition (Forward secrecy)

For every PPT interactive adversary A,

$$\left|\Pr[A^{\mathsf{Real}}(1^\ell) = 1] - \Pr[A^{\mathsf{Ideal}}(1^\ell) = 1]\right| \leq \mathrm{negl}(\ell).$$

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Ideal Ratchet

$c :=$ false; $R := \perp$.

Advance():

if not c: $K \leftarrow \mathsf{U}_\ell$.

else: $R \| K := G(R)$.

return K.

Compromise():

if not c: $R \leftarrow \mathsf{U}_\ell$.

$c :=$ true; return R.

Real Ratchet

$R \leftarrow \mathsf{U}_\ell$.

Advance():
$R \| K := G(R)$.
return K.

Compromise():
return R.

Ideal Ratchet

$c :=$ false; $R := \bot$.

Advance():
if not c: $K \leftarrow \mathsf{U}_\ell$.
else: $R \| K := G(R)$.
return K.

Compromise():
if not c: $R \leftarrow \mathsf{U}_\ell$.
$c :=$ true; return R.

Proof goal

$$\text{Real} \approx_c \text{Ideal}.$$

Move between the two ratchets through indistinguishable intermediate games.

1. $H_0 \to H_1$: sample the initial state lazily.

1. $H_0 \to H_1$: sample the initial state lazily.
2. $H_1 \to H_2$: split the Advance code into two branches.

1. $H_0 \to H_1$: sample the initial state lazily.
2. $H_1 \to H_2$: split the Advance code into two branches.
3. $H_{3,0} \to \cdots \to H_{3,Q}$: replace one update at a time using PRG security.

1. $H_0 \to H_1$: sample the initial state lazily.
2. $H_1 \to H_2$: split the Advance code into two branches.
3. $H_{3,0} \to \cdots \to H_{3,Q}$: replace one update at a time using PRG security.
4. $H_{3,Q} \to H_4$: defer sampling the unused state.

1. $H_0 \to H_1$: sample the initial state lazily.
2. $H_1 \to H_2$: split the Advance code into two branches.
3. $H_{3,0} \to \cdots \to H_{3,Q}$: replace one update at a time using PRG security.
4. $H_{3,Q} \to H_4$: defer sampling the unused state.

Proof path

$$\text{Real} = H_0 \equiv H_1 \equiv H_2 \equiv H_{3,0} \approx_c \cdots \approx_c H_{3,Q} \equiv H_4 = \text{Ideal}.$$

Challenger H_0

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Challenger H_0

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Challenger H_1

$R := \perp$.

Advance():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.

$R \| K := G(R)$.

return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.

return R.

Challenger H_0

$R \leftarrow \mathsf{U}_\ell$.

Advance():

$R \| K := G(R)$.

return K.

Compromise():

return R.

Challenger H_1

$R := \perp$.

Advance():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.

$R \| K := G(R)$.

return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.

return R.

Why $H_0 \equiv H_1$ (identical distributions). No output depends on R before its first use. Sampling it then gives the same uniform state; every later query has the same response.

Challenger H_1

$R := \perp$.

Advance():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger H_1

$R := \bot$.

Advance():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger H_2

$R := \bot$.

Advance():

if $R = \bot$:
  $R \leftarrow \mathsf{U}_\ell$.
  $R \| K := G(R)$.
else: $R \| K := G(R)$.
return K.

Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger H_1

$R := \bot$.

Advance():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger H_2

$R := \bot$.

Advance():
if $R = \bot$:
$R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
else: $R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Why $H_1 \equiv H_2$ (identical behavior). For either value of the condition, both programs sample the same values, compute the same update, and return the same key or state.

Let $Q = Q(\ell)$ bound the PPT adversary's Advance calls. Queries remain adaptive.

Challenger $H_{3,0}$

$R := \bot$.

Advance():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Let $Q = Q(\ell)$ bound the PPT adversary's Advance calls. Queries remain adaptive.

Challenger $H_{3,0}$

$R := \bot$.

Advance():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger $H_{3,Q}$

$R := \bot$; $c :=$ false.

Advance():
if not c: $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else: $R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$c :=$ true; return R.

Let $Q = Q(\ell)$ bound the PPT adversary's Advance calls. Queries remain adaptive.

Challenger $H_{3,0}$

$R := \bot$.

Advance():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger $H_{3,Q}$

$R := \bot$; $c :=$ false.

Advance():
if not c: $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else: $R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$c :=$ true; return R.

We are not only changing the flag: we are implicitly replacing the PRG output with a uniform string in every pre-compromise Advance query. PRG security lets us replace only one use at a time, so we need one sub-hybrid for each use.

Let $Q = Q(\ell)$ bound the number of Advance calls made by the PPT adversary.

Let $Q = Q(\ell)$ bound the number of Advance calls made by the PPT adversary.

Hybrid $H_{3,i}$, for $0 \leq i \leq Q$

On the first i advances before compromise, sample $(R, K) \leftarrow \mathsf{U}_{2\ell}$. On all other advances, compute $R \| K := G(R)$.

Let $Q = Q(\ell)$ bound the number of Advance calls made by the PPT adversary.

Hybrid $H_{3,i}$, for $0 \le i \le Q$

On the first i advances before compromise, sample $(R, K) \leftarrow \mathsf{U}_{2\ell}$. On all other advances, compute $R \| K := G(R)$.

- $H_{3,0} \equiv H_2$: every update uses G.
- $H_{3,1}$ replaces only the first pre-compromise update.
- $H_{3,Q}$ replaces every pre-compromise update.

Challenger $H_{3,0}$

$R := \bot$.

Advance():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$R \| K := G(R)$.
return K.

Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
return R.

Challenger $H_{3,0}$

$R := \bot$.

Advance():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  $R \| K := G(R)$.
  return K.

Compromise():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  return R.

$H_{3,0}$ with c and t

$R := \bot$; $c :=$ false; $t := 0$.

Advance():
  $t := t + 1$.
  if $\neg c$ and $t \leq 0$:
    $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
  else:
    if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
    $R \| K := G(R)$.
  return K.

Compromise():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  $c :=$ true; return R.

Challenger $H_{3,0}$

$R := \bot$.

Advance():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  $R \| K := G(R)$.
  return K.

Compromise():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  return R.

$H_{3,0}$ with c and t

$R := \bot$; $c :=$ false; $t := 0$.

Advance():
  $t := t + 1$.
  if $\neg c$ and $t \leq 0$:
    $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
  else:
    if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
    $R \| K := G(R)$.
  return K.

Compromise():
  if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
  $c :=$ true; return R.

Identical behavior. The new variables only record queries. Since $t \geq 1$ during Advance, the condition $t \leq 0$ never holds: every update still uses G.

Challenger $H_{3,i}$

$R := \perp$; $c :=$ false; $t := 0$.

Advance():

$t := t + 1$.
if $\neg c$ and $t \leq i$:
  $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else:
  if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
  $R \| K := G(R)$.
return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Challenger $H_{3,i}$

$R := \perp$; $c :=$ false; $t := 0$.

Advance():

$t := t + 1$.
if $\neg c$ and $t \leq i$:
  $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else:
  if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
  $R \| K := G(R)$.
return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Challenger $H_{3,i+1}$

$R := \perp$; $c :=$ false; $t := 0$.

Advance():

$t := t + 1$.
if $\neg c$ and $t \leq i + 1$:
  $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else:
  if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
  $R \| K := G(R)$.
return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Challenger $H_{3,i}$

$R := \bot$; $c :=$ false; $t := 0$.

Advance():

$t := t + 1$.
if $\neg c$ and $t \leq i$:
  $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else:
  if $R = \bot$: $R \leftarrow \mathsf{U}_{\ell}$.
  $R \| K := G(R)$.
return K.

Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Challenger $H_{3,i+1}$

$R := \bot$; $c :=$ false; $t := 0$.

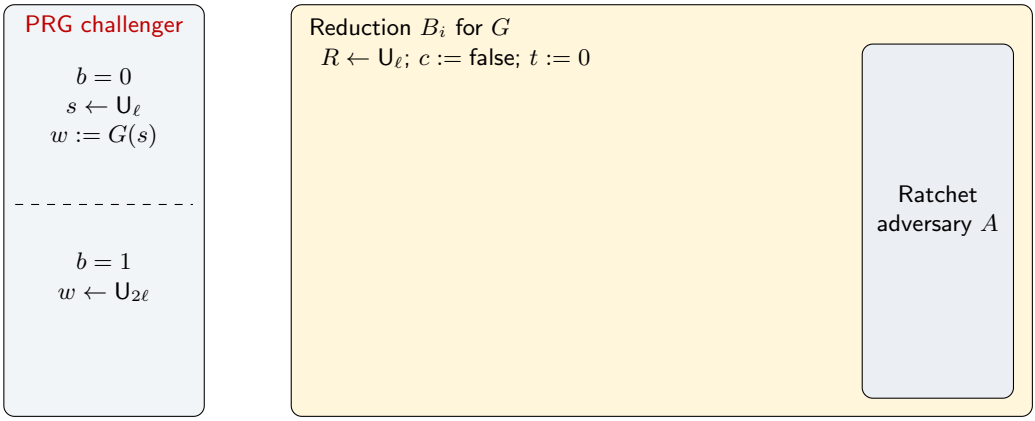
Advance():

$t := t + 1$.
if $\neg c$ and $t \leq i + 1$:
  $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else:
  if $R = \bot$: $R \leftarrow \mathsf{U}_{\ell}$.
  $R \| K := G(R)$.
return K.

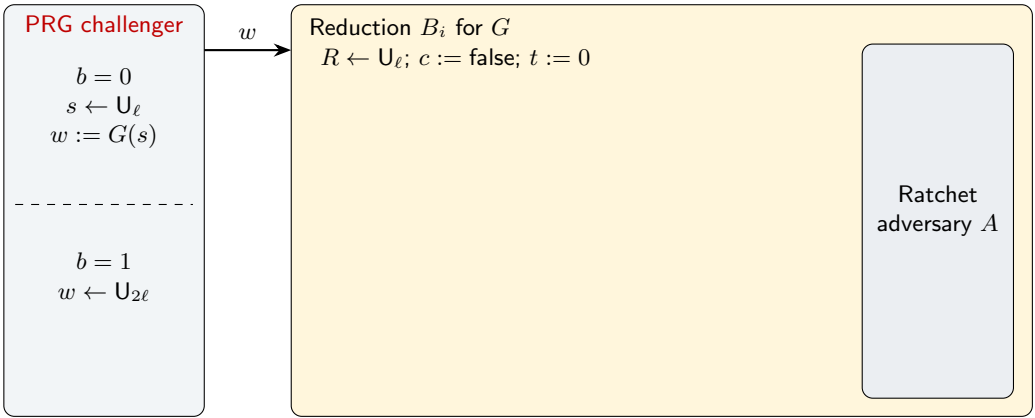
Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

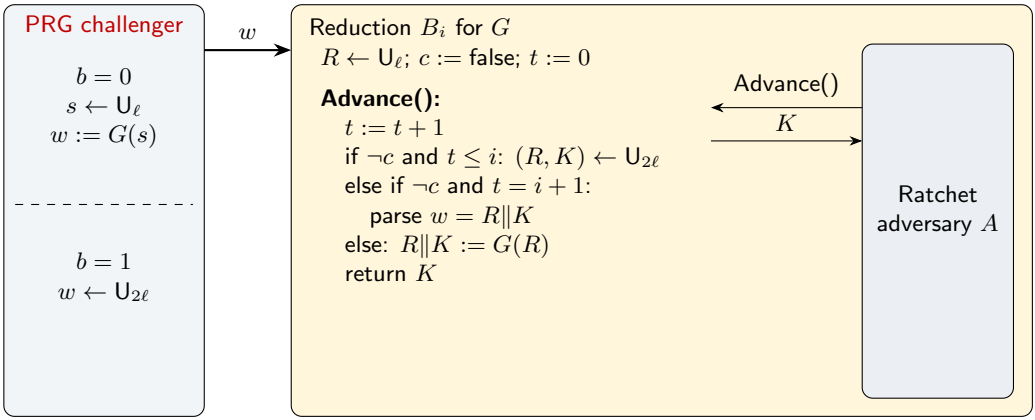
Why $H_{3,i} \approx_c H_{3,i+1}$ (PRG security). Only advance $i + 1$ changes, if it precedes compromise: $G(R)$ becomes a uniform pair. All earlier and later calls look the same.


PRG challenger
$b = 0$
$s \leftarrow \mathsf{U}_\ell$
$w := G(s)$
$b = 1$
$w \leftarrow \mathsf{U}_{2\ell}$
Reduction B_i for G
$R \leftarrow \mathsf{U}_\ell$; $c :=$ false; $t := 0$
Ratchet adversary A

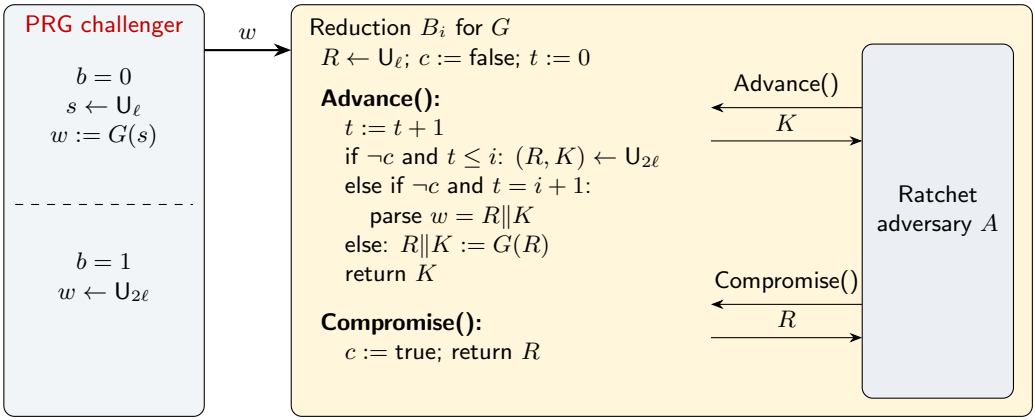
Step 3: Reduction to PRG Security



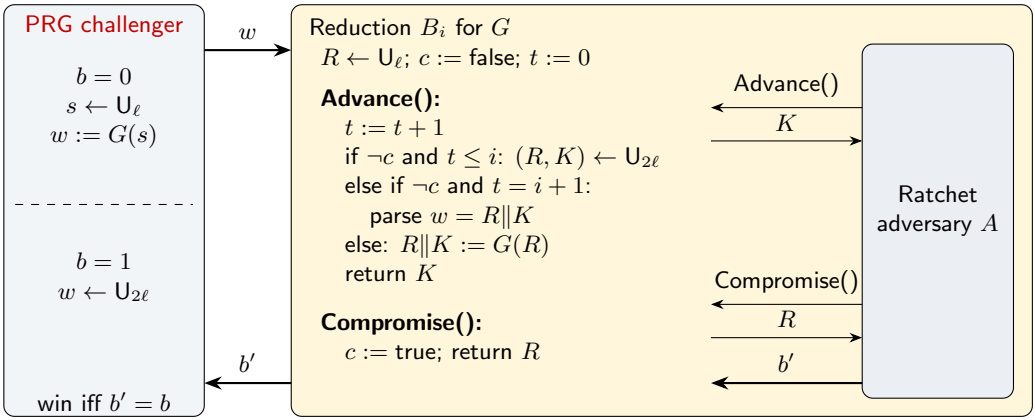
Step 3: Reduction to PRG Security



Step 3: Reduction to PRG Security



Step 3: Reduction to PRG Security



Case 1: $b = 1$ — truly random

At advance $i + 1$, $w \leftarrow \mathsf{U}_{2\ell}$ supplies an independent state and key. The first $i + 1$ pre-compromise updates are uniform: A interacts with $H_{3,i+1}$.

Case 1: $b = 1$ — truly random

At advance $i + 1$, $w \leftarrow \mathsf{U}_{2\ell}$ supplies an independent state and key. The first $i + 1$ pre-compromise updates are uniform: A interacts with $H_{3,i+1}$.

Case 2: $b = 0$ — pseudorandom

At advance $i + 1$, $w = G(s)$ for $s \leftarrow \mathsf{U}_\ell$ replaces $G(R)$. The hidden state R was uniform and independent of the earlier replies: A interacts with $H_{3,i}$.

Case 1: $b = 1$ — truly random

At advance $i + 1$, $w \leftarrow \mathsf{U}_{2\ell}$ supplies an independent state and key. The first $i + 1$ pre-compromise updates are uniform: A interacts with $H_{3,i+1}$.

Case 2: $b = 0$ — pseudorandom

At advance $i + 1$, $w = G(s)$ for $s \leftarrow \mathsf{U}_\ell$ replaces $G(R)$. The hidden state R was uniform and independent of the earlier replies: A interacts with $H_{3,i}$.

If compromise comes first, the two simulations are identical and w is unused. Let $p_i = \Pr[A^{H_{3,i}} = 1]$. Since B_i forwards b',

$$\Pr[\mathsf{PRG}_{B_i,G}(\ell) = 1] = \frac{1}{2}(1 - p_i) + \frac{1}{2}p_{i+1}.$$

Step 3: Proving the Bigger Step from the Sub-Hybrids

We proved each neighboring sub-hybrid indistinguishable using B_i. Now use these reductions to build one PRG adversary B for the endpoint gap.

We proved each neighboring sub-hybrid indistinguishable using B_i. Now use these reductions to build one PRG adversary B for the endpoint gap.

Suppose A distinguishes $H_{3,0}$ and $H_{3,Q}$

Assume without loss of generality that, for non-negligible $\epsilon(\ell)$,

$$p_Q - p_0 = \Pr[A^{H_{3,Q}} = 1] - \Pr[A^{H_{3,0}} = 1] \geq \epsilon(\ell).$$

We proved each neighboring sub-hybrid indistinguishable using B_i. Now use these reductions to build one PRG adversary B for the endpoint gap.

Suppose A distinguishes $H_{3,0}$ and $H_{3,Q}$

Assume without loss of generality that, for non-negligible $\epsilon(\ell)$,

$$p_Q - p_0 = \Pr[A^{H_{3,Q}} = 1] - \Pr[A^{H_{3,0}} = 1] \geq \epsilon(\ell).$$

Construct B using the reductions B_i

Given a PRG challenge w:

1. Choose I uniformly from $\{0, \ldots, Q-1\}$.

Step 3: Proving the Bigger Step from the Sub-Hybrids

We proved each neighboring sub-hybrid indistinguishable using B_i. Now use these reductions to build one PRG adversary B for the endpoint gap.

Suppose A distinguishes $H_{3,0}$ and $H_{3,Q}$

Assume without loss of generality that, for non-negligible $\epsilon(\ell)$,

$$p_Q - p_0 = \Pr[A^{H_{3,Q}} = 1] - \Pr[A^{H_{3,0}} = 1] \geq \epsilon(\ell).$$

Construct B using the reductions B_i

Given a PRG challenge w:

1. Choose I uniformly from $\{0, \ldots, Q-1\}$.
2. Run B_I with challenge w and adversary A.

We proved each neighboring sub-hybrid indistinguishable using B_i. Now use these reductions to build one PRG adversary B for the endpoint gap.

Suppose A distinguishes $H_{3,0}$ and $H_{3,Q}$

Assume without loss of generality that, for non-negligible $\epsilon(\ell)$,

$$p_Q - p_0 = \Pr[A^{H_{3,Q}} = 1] - \Pr[A^{H_{3,0}} = 1] \geq \epsilon(\ell).$$

Construct B using the reductions B_i

Given a PRG challenge w:

1. Choose I uniformly from $\{0, \ldots, Q-1\}$.
2. Run B_I with challenge w and adversary A.
3. Forward A's bit b' as the PRG guess.

Step 3: The Random-Index Reduction

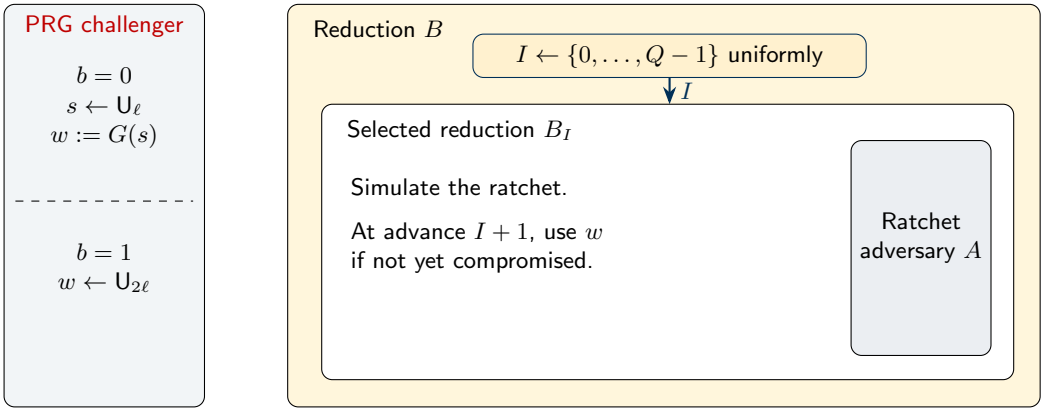
PRG challenger

$b = 0$
$s \leftarrow \mathsf{U}_\ell$
$w := G(s)$

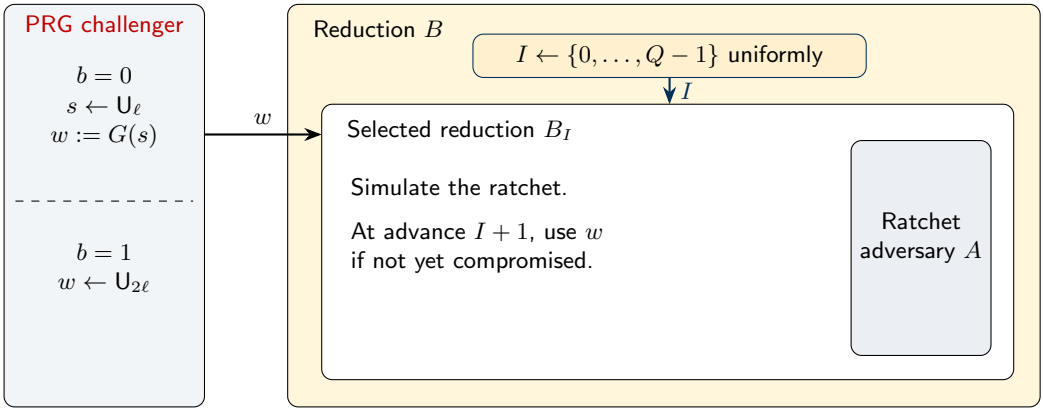
$b = 1$
$w \leftarrow \mathsf{U}_{2\ell}$

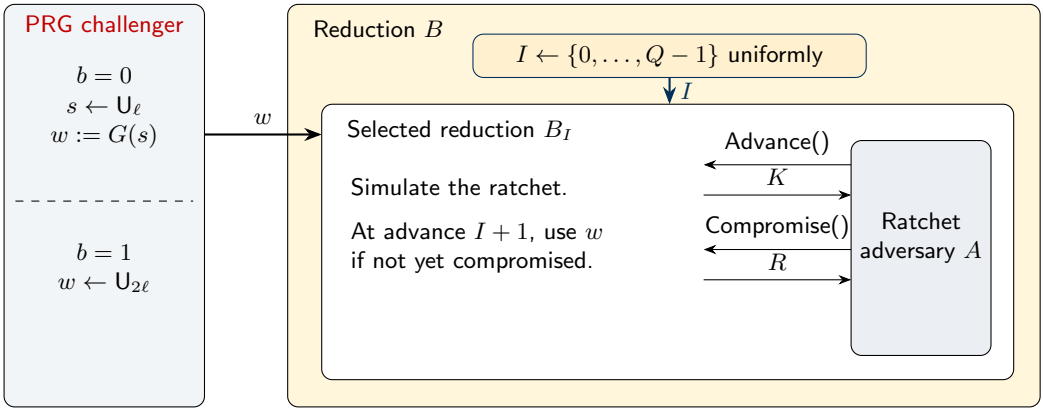
Reduction B

Step 3: The Random-Index Reduction

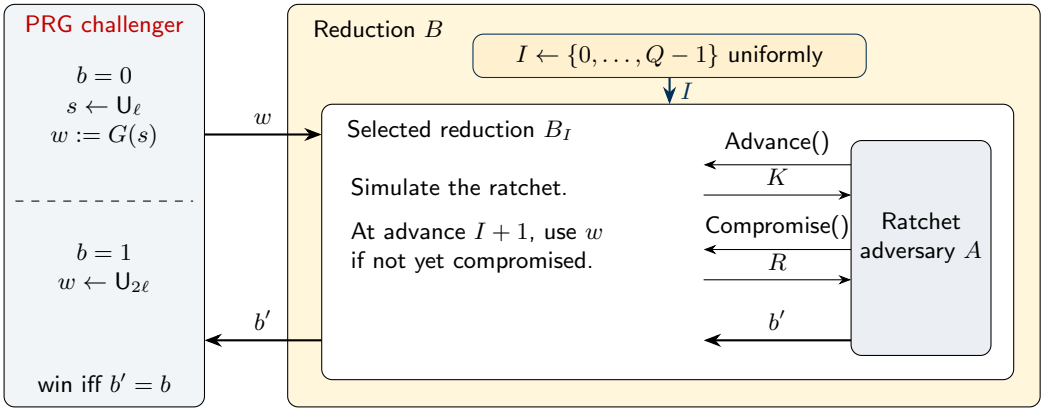


Step 3: The Random-Index Reduction




PRG challenger
$b = 0$
$s \leftarrow \mathsf{U}_\ell$
$w := G(s)$
$b = 1$
$w \leftarrow \mathsf{U}_{2\ell}$
w
Reduction B
$I \leftarrow \{0, \ldots, Q-1\}$ uniformly
I
Selected reduction B_I
Simulate the ratchet.
At advance $I+1$, use w if not yet compromised.
Advance()
K
Compromise()
R
Ratchet adversary A

Step 3: The Random-Index Reduction



Step 3: Success for a Fixed Index

Let $\delta_i = p_{i+1} - p_i$. Since $\sum_{i=0}^{Q-1} \delta_i = p_Q - p_0 \geq \epsilon(\ell)$, at least one $\delta_i \geq \epsilon(\ell)/Q$.

Step 3: Success for a Fixed Index

Let $\delta_i = p_{i+1} - p_i$. Since $\sum_{i=0}^{Q-1} \delta_i = p_Q - p_0 \geq \epsilon(\ell)$, at least one $\delta_i \geq \epsilon(\ell)/Q$.

Condition on the sampled index $I = i$

B runs B_i and forwards A's bit b'.

- If $b = 0$: A interacts with $H_{3,i}$.
  B wins when A outputs 0, with probability $1 - p_i$.

Step 3: Success for a Fixed Index

Let $\delta_i = p_{i+1} - p_i$. Since $\sum_{i=0}^{Q-1} \delta_i = p_Q - p_0 \geq \epsilon(\ell)$, at least one $\delta_i \geq \epsilon(\ell)/Q$.

Condition on the sampled index $I = i$

B runs B_i and forwards A's bit b'.

- If $b = 0$: A interacts with $H_{3,i}$.
  B wins when A outputs 0, with probability $1 - p_i$.
- If $b = 1$: A interacts with $H_{3,i+1}$.
  B wins when A outputs 1, with probability p_{i+1}.

Let $\delta_i = p_{i+1} - p_i$. Since $\sum_{i=0}^{Q-1} \delta_i = p_Q - p_0 \geq \epsilon(\ell)$, at least one $\delta_i \geq \epsilon(\ell)/Q$.

Condition on the sampled index $I = i$

B runs B_i and forwards A's bit b'.

- If $b = 0$: A interacts with $H_{3,i}$.
  B wins when A outputs 0, with probability $1 - p_i$.
- If $b = 1$: A interacts with $H_{3,i+1}$.
  B wins when A outputs 1, with probability p_{i+1}.

The PRG challenge bit is uniform and independent of I

$$\begin{aligned}\Pr[B \text{ wins} \mid I = i] &= \frac{1}{2}(1 - p_i) + \frac{1}{2}p_{i+1} \\ &= \frac{1}{2} + \frac{p_{i+1} - p_i}{2} = \frac{1}{2} + \frac{\delta_i}{2}.\end{aligned}$$

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Condition on which index was sampled

$$\Pr[B \text{ wins}] = \sum_{i=0}^{Q-1} \Pr[I = i] \Pr[B \text{ wins} \mid I = i]$$

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Condition on which index was sampled

$$\begin{aligned}\Pr[B \text{ wins}] &= \sum_{i=0}^{Q-1} \Pr[I = i] \Pr[B \text{ wins} \mid I = i] \\ &= \sum_{i=0}^{Q-1} \frac{1}{Q} \left(\frac{1}{2} + \frac{\delta_i}{2} \right)\end{aligned}$$

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Condition on which index was sampled

$$\begin{aligned}
\Pr[B \text{ wins}] &= \sum_{i=0}^{Q-1} \Pr[I = i] \Pr[B \text{ wins} \mid I = i] \\
&= \sum_{i=0}^{Q-1} \frac{1}{Q} \left(\frac{1}{2} + \frac{\delta_i}{2} \right) \\
&= \frac{1}{2} + \frac{1}{2Q} \sum_{i=0}^{Q-1} \delta_i
\end{aligned}$$

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Condition on which index was sampled

$$\begin{aligned}
\Pr[B \text{ wins}] &= \sum_{i=0}^{Q-1} \Pr[I = i] \Pr[B \text{ wins} \mid I = i] \\
&= \sum_{i=0}^{Q-1} \frac{1}{Q} \left(\frac{1}{2} + \frac{\delta_i}{2} \right) \\
&= \frac{1}{2} + \frac{1}{2Q} \sum_{i=0}^{Q-1} \delta_i \\
&= \frac{1}{2} + \frac{p_Q - p_0}{2Q} \geq \frac{1}{2} + \frac{\epsilon(\ell)}{2Q}.
\end{aligned}$$

Step 3: Bounding the Advantage of B

B samples I uniformly from Q possible indices, so $\Pr[I = i] = 1/Q$ for every $i \in \{0, \ldots, Q-1\}$.

Condition on which index was sampled

$$\begin{aligned}
\Pr[B \text{ wins}] &= \sum_{i=0}^{Q-1} \Pr[I = i] \Pr[B \text{ wins} \mid I = i] \\
&= \sum_{i=0}^{Q-1} \frac{1}{Q} \left(\frac{1}{2} + \frac{\delta_i}{2} \right) \\
&= \frac{1}{2} + \frac{1}{2Q} \sum_{i=0}^{Q-1} \delta_i \\
&= \frac{1}{2} + \frac{p_Q - p_0}{2Q} \geq \frac{1}{2} + \frac{\epsilon(\ell)}{2Q}.
\end{aligned}$$

Since Q is polynomial, $\epsilon(\ell)/(2Q)$ is non-negligible. This contradicts PRG security, so $\boxed{H_{3,0} \approx_c H_{3,Q}}$.

Challenger $H_{3,Q}$

$R := \perp$; $c :=$ false.

Advance():

if not c: $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else: $R \| K := G(R)$.
return K.

Compromise():

if $R = \perp$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Challenger $H_{3,Q}$

$R := \bot$; $c :=$ false.

Advance():

if not c: $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else: $R \| K := G(R)$.
return K.

Compromise():

if $R = \bot$: $R \leftarrow \mathsf{U}_\ell$.
$c :=$ true; return R.

Ideal Ratchet H_4

$c :=$ false; $R := \bot$.

Advance():

if not c: $K \leftarrow \mathsf{U}_\ell$.
else: $R \| K := G(R)$.
return K.

Compromise():

if not c: $R \leftarrow \mathsf{U}_\ell$.
$c :=$ true; return R.

Challenger $H_{3,Q}$

$R := \bot$; $c :=$ false.

Advance():
if not c: $(R, K) \leftarrow \mathsf{U}_{2\ell}$.
else: $R \| K := G(R)$.
return K.

Compromise():
if $R = \bot$: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Ideal Ratchet H_4

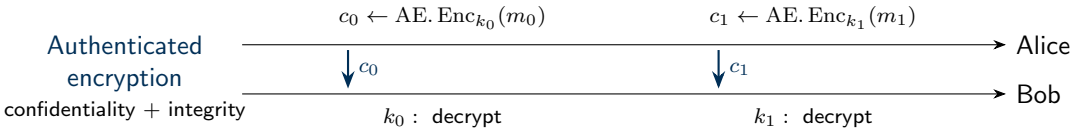
$c :=$ false; $R := \bot$.

Advance():
if not c: $K \leftarrow \mathsf{U}_{\ell}$.
else: $R \| K := G(R)$.
return K.

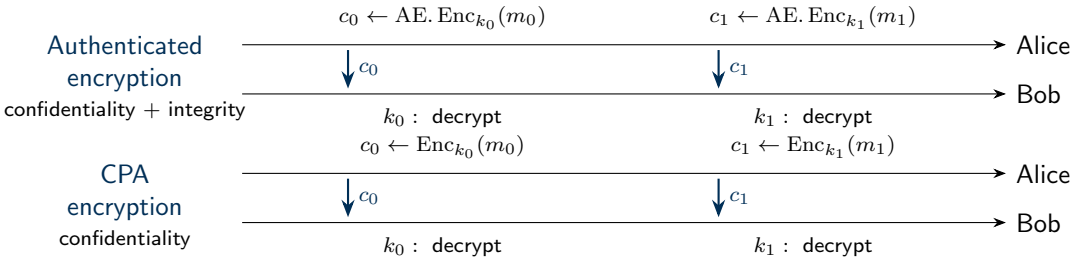
Compromise():
if not c: $R \leftarrow \mathsf{U}_{\ell}$.
$c :=$ true; return R.

Why $H_{3,Q} \equiv H_4$ (identical distributions). Before compromise, every advance overwrites R without reading it. Only K is returned. Sample R at the first compromise instead; afterward, both games use G normally.

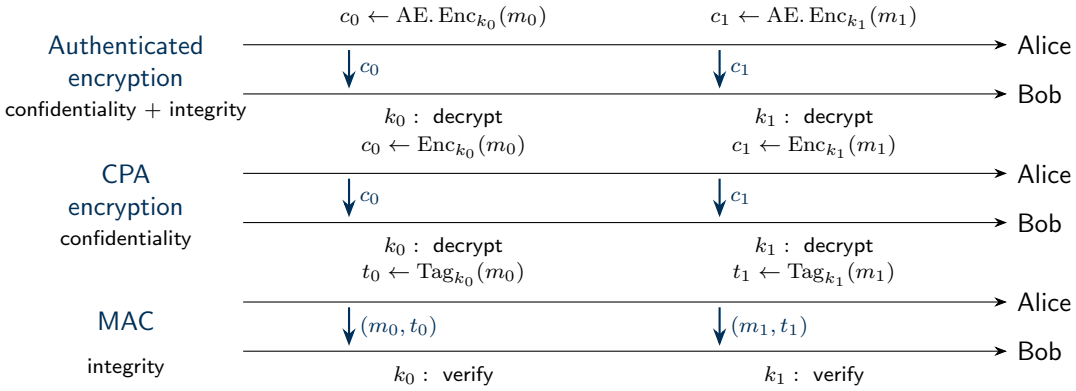
Ratchet Keys Can Be Used Beyond AE



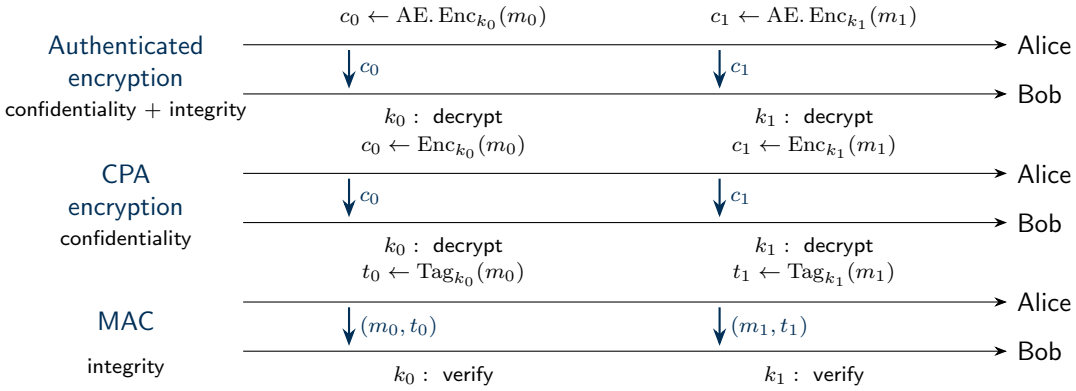
Ratchet Keys Can Be Used Beyond AE



Ratchet Keys Can Be Used Beyond AE



Ratchet Keys Can Be Used Beyond AE



The ratchet supplies shared pseudorandom keys $k_0, k_1, \ldots$.

After a compromise

The adversary has the same current secret material as the parties. Can Alice and Bob agree on a new key that the adversary cannot compute?

After a compromise

The adversary has the same current secret material as the parties. Can Alice and Bob agree on a new key that the adversary cannot compute?

The symmetric ratchet cannot do this alone

From st_i, the adversary computes $G(\mathsf{st}_i)$ and every later state and key. Deterministic updates do not create a new secret.

After a compromise

The adversary has the same current secret material as the parties. Can Alice and Bob agree on a new key that the adversary cannot compute?

The symmetric ratchet cannot do this alone

From st_i, the adversary computes $G(\mathsf{st}_i)$ and every later state and key. Deterministic updates do not create a new secret.

We need fresh secret input that the adversary does not know.

We Have Already Seen This: Key Agreement

- Initially, the parties have no shared secret unknown to the adversary.

We Have Already Seen This: Key Agreement

- Initially, the parties have no shared secret unknown to the adversary.
- Alice and Bob sample fresh private randomness and exchange public values.

- Initially, the parties have no shared secret unknown to the adversary.
- Alice and Bob sample fresh private randomness and exchange public values.
- They derive a shared key that an eavesdropper cannot compute.

Diffie–Hellman

$$A = g^a, \qquad B = g^b, \qquad k = \mathrm{KDF}(g^{ab}).$$

- Initially, the parties have no shared secret unknown to the adversary.
- Alice and Bob sample fresh private randomness and exchange public values.
- They derive a shared key that an eavesdropper cannot compute.

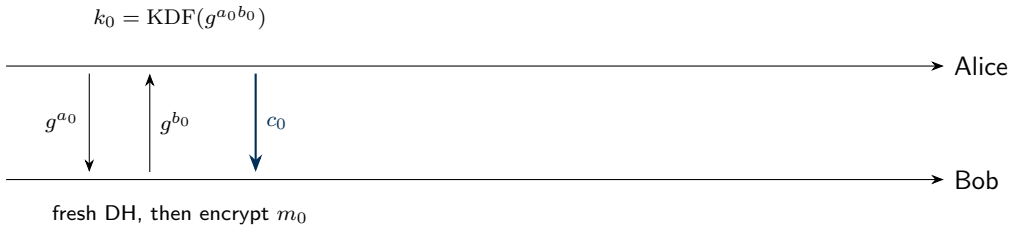
Diffie–Hellman

$$A = g^a, \qquad B = g^b, \qquad k = \mathrm{KDF}(g^{ab}).$$

We will use this to continuously refresh the messaging keys.

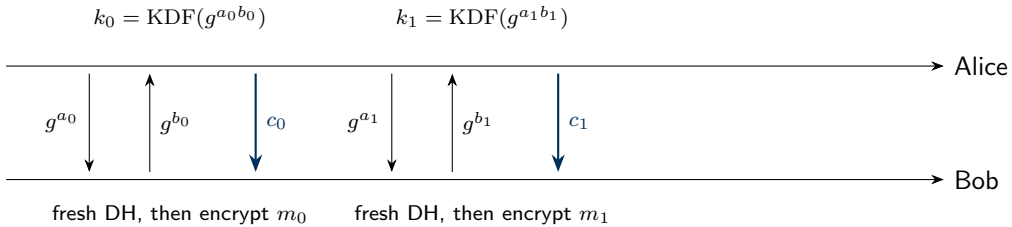
Let's start by seeing how to do this when only Alice is sending messages to Bob.

Option 1: Fresh DH for Every Message



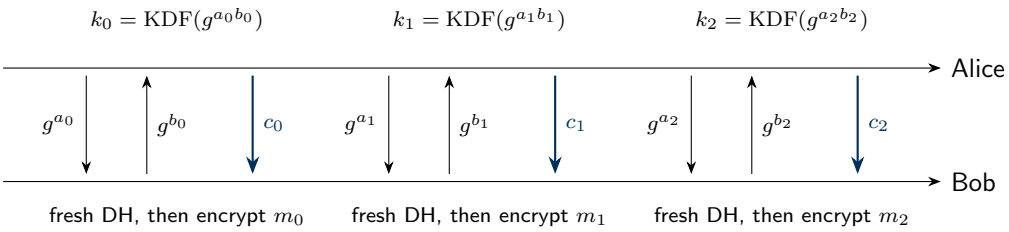
Each c_i encrypts Alice's m_i with the fresh shared k_i.

Option 1: Fresh DH for Every Message



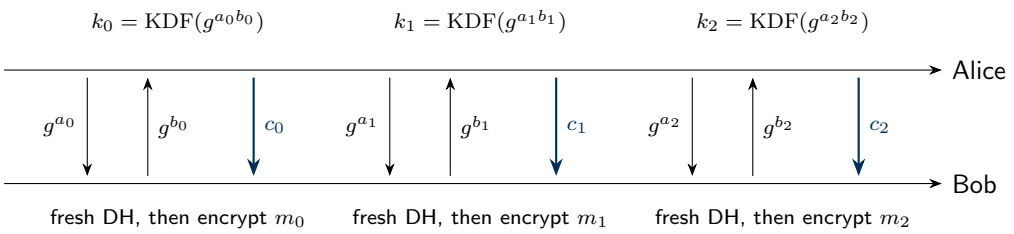
Each c_i encrypts Alice's m_i with the fresh shared k_i.

Option 1: Fresh DH for Every Message



Each c_i encrypts Alice's m_i with the fresh shared k_i.

Option 1: Fresh DH for Every Message

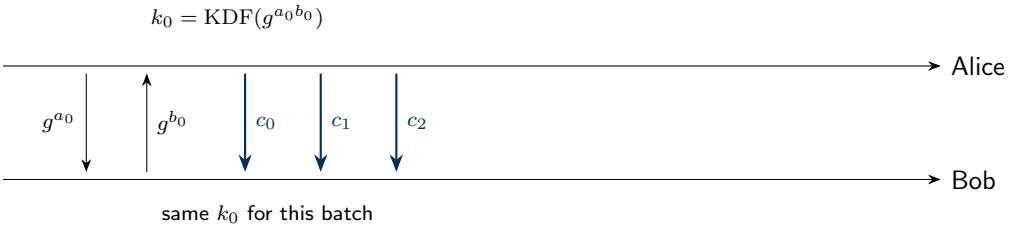


Each c_i encrypts Alice's m_i with the fresh shared k_i.

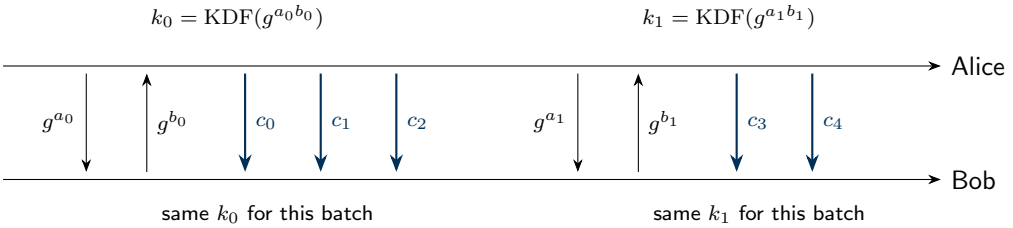
Problems

Interactivity for every message sent.

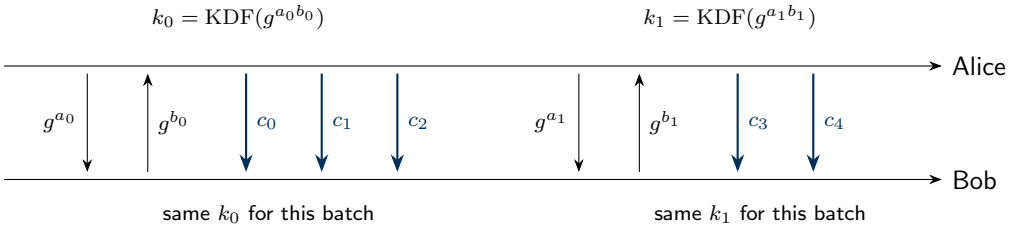
Option 2: Several Messages per DH Key



Option 2: Several Messages per DH Key



Option 2: Several Messages per DH Key



Tradeoff

Fewer DH exchanges, but exposing a batch key exposes all messages in that batch.

Option 3: Ratcheting Between DH Exchanges

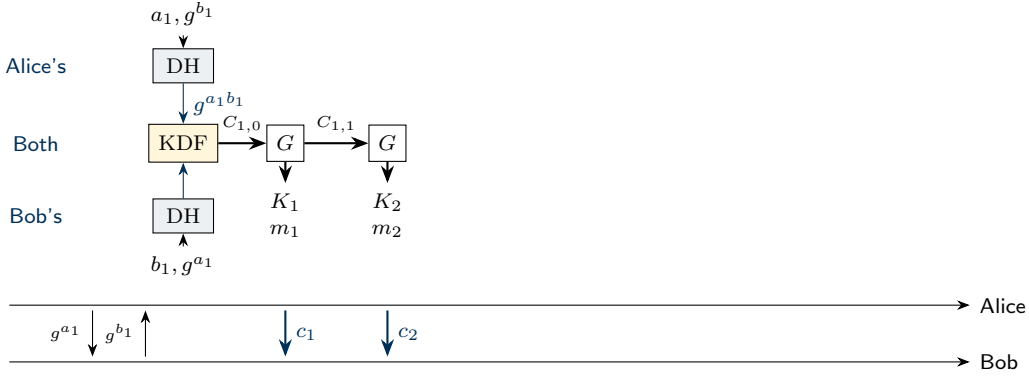


Option 3: Ratcheting Between DH Exchanges



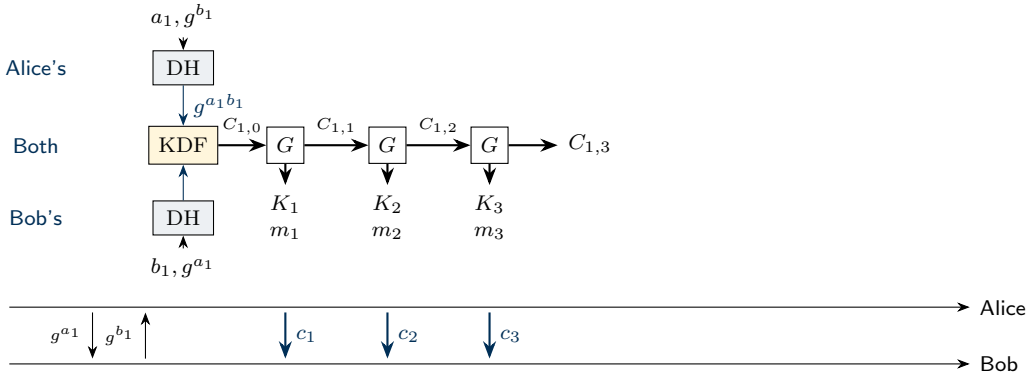
A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

Option 3: Ratcheting Between DH Exchanges



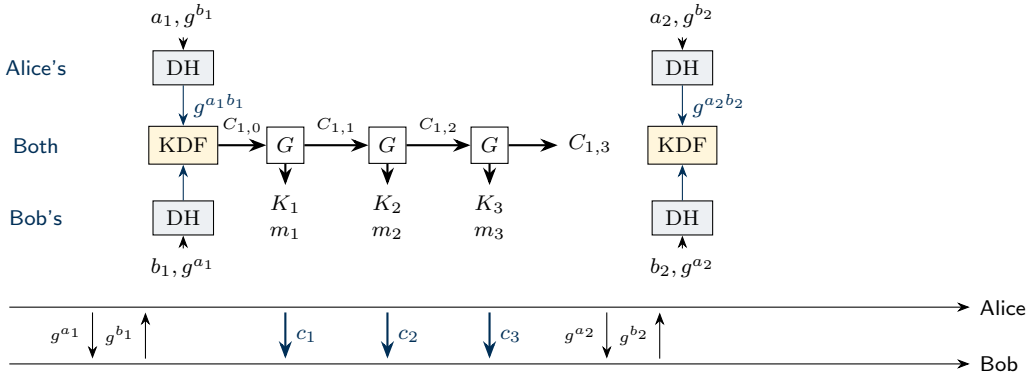
A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

Option 3: Ratcheting Between DH Exchanges



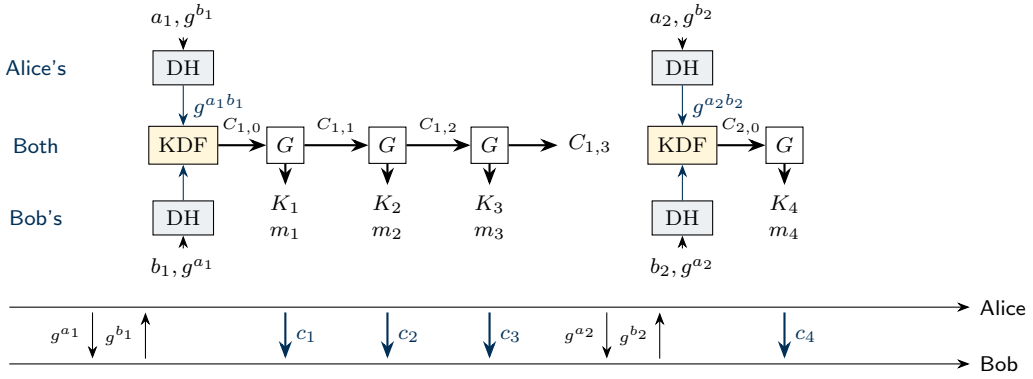
A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

Option 3: Ratcheting Between DH Exchanges



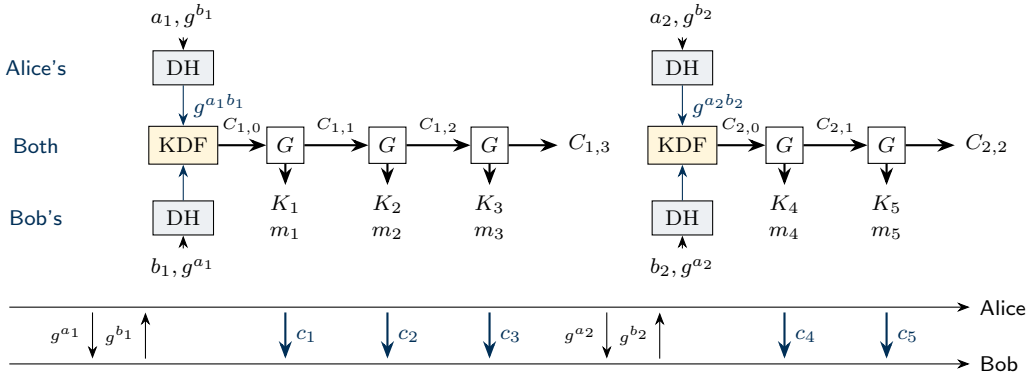
A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

Option 3: Ratcheting Between DH Exchanges



A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

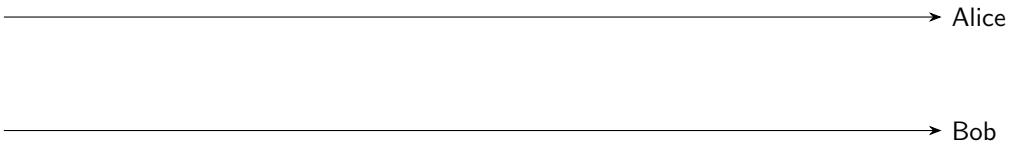
Option 3: Ratcheting Between DH Exchanges



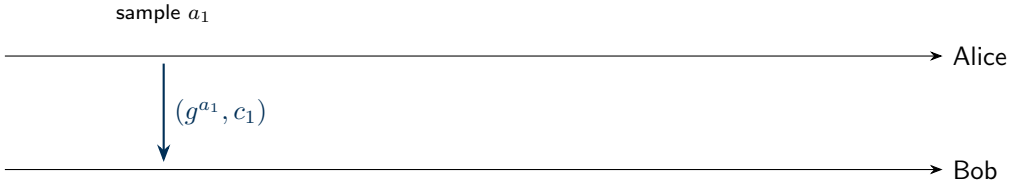
A symmetric tick gives K_i; the matching message is $c_i = \mathrm{AE.Enc}_{K_i}(m_i)$.

Both sending messages

Bob's current public value is g^{b_0}; both parties share the ratchet state.



Bob's current public value is g^{b_0}; both parties share the ratchet state.

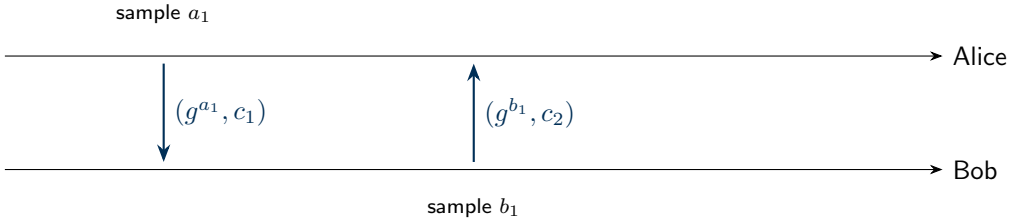


Attach the DH public value to the message

$$R_i \| C_{i,0} := G(R_{i-1}, X_i), \qquad c_i = \mathrm{AE.Enc}_{K_i}(m_i).$$

Derive K_i from the newly initialized message chain $C_{i,0}$.

Bob's current public value is g^{b_0}; both parties share the ratchet state.

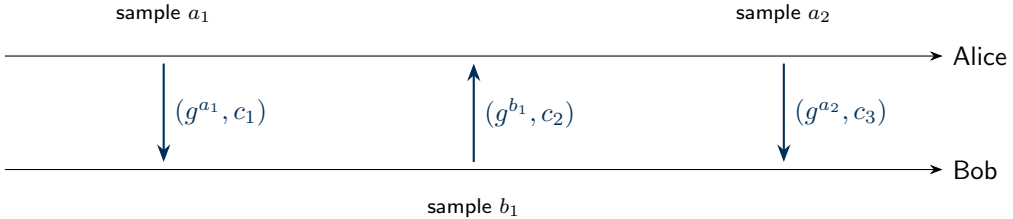


Attach the DH public value to the message

$$R_i \| C_{i,0} := G(R_{i-1}, X_i), \qquad c_i = \mathrm{AE.Enc}_{K_i}(m_i).$$

Derive K_i from the newly initialized message chain $C_{i,0}$.

Bob's current public value is g^{b_0}; both parties share the ratchet state.

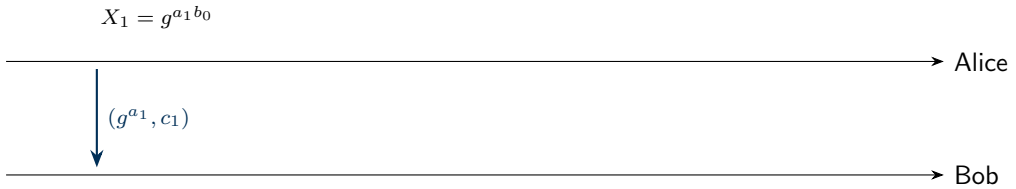


Attach the DH public value to the message

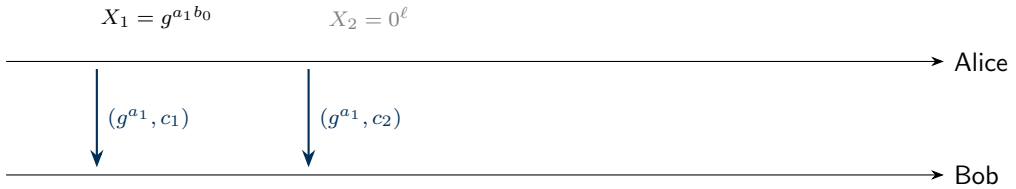
$$R_i \| C_{i,0} := G(R_{i-1}, X_i), \qquad c_i = \mathrm{AE.Enc}_{K_i}(m_i).$$

Derive K_i from the newly initialized message chain $C_{i,0}$.

Consecutive Messages from Alice



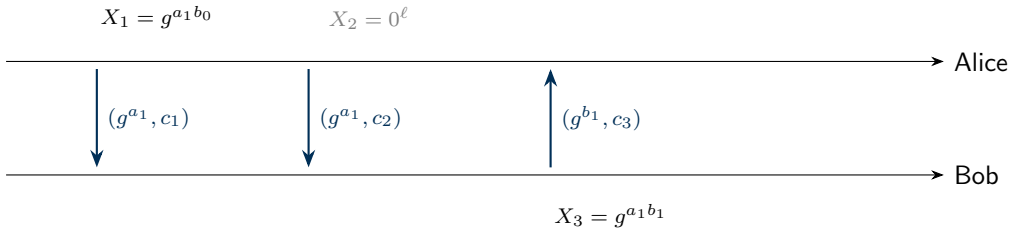
Consecutive Messages from Alice



Same DH public value; a new message key

Alice keeps g^{a_1} for consecutive sends but advances the symmetric ratchet for every message: $K_1 \neq K_2$ except with negligible probability.

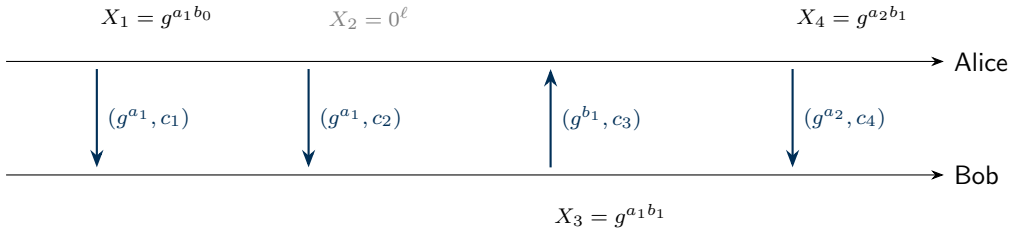
Consecutive Messages from Alice



Same DH public value; a new message key

Alice keeps g^{a_1} for consecutive sends but advances the symmetric ratchet for every message: $K_1 \neq K_2$ except with negligible probability.

Consecutive Messages from Alice



Same DH public value; a new message key

Alice keeps g^{a_1} for consecutive sends but advances the symmetric ratchet for every message: $K_1 \neq K_2$ except with negligible probability.

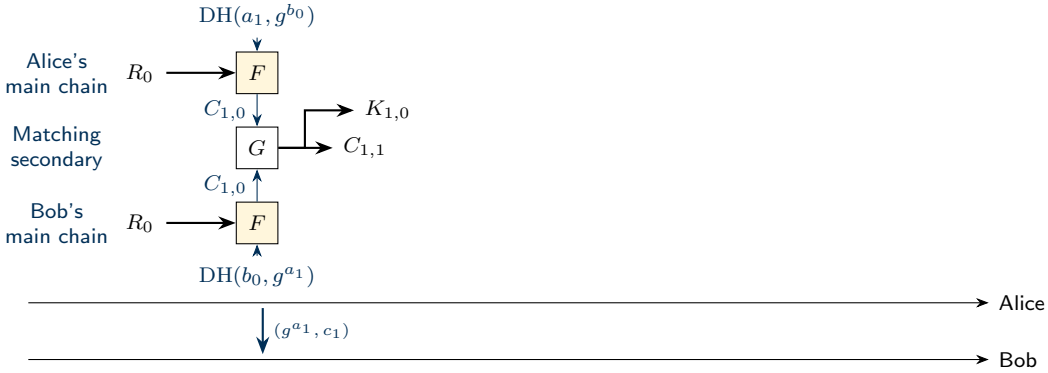
A new sending turn introduces a fresh DH public value.

Combining the Ratchets



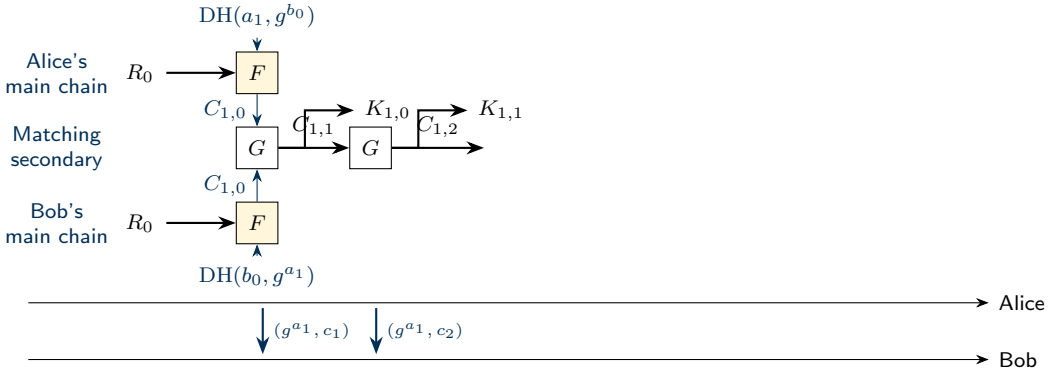
Initial DH update seeds the Alice-to-Bob secondary chain.

Combining the Ratchets



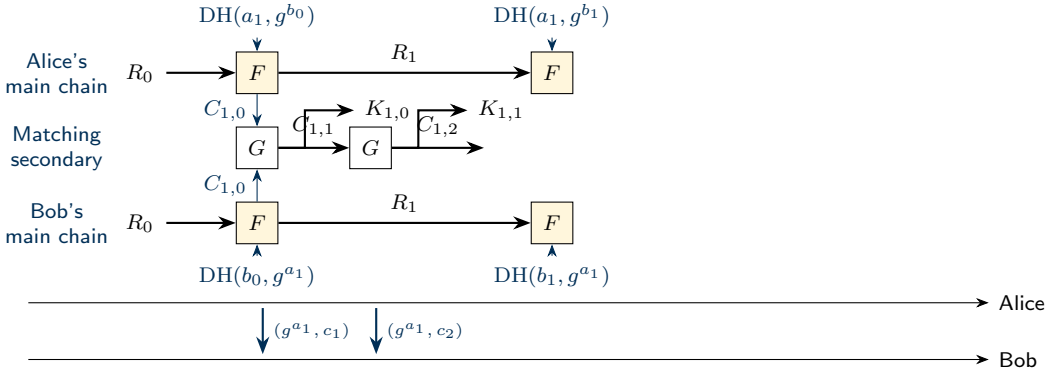
First tick: $K_{1,0}$ encrypts m_1; Alice sends (g^{a_1}, c_1).

Combining the Ratchets



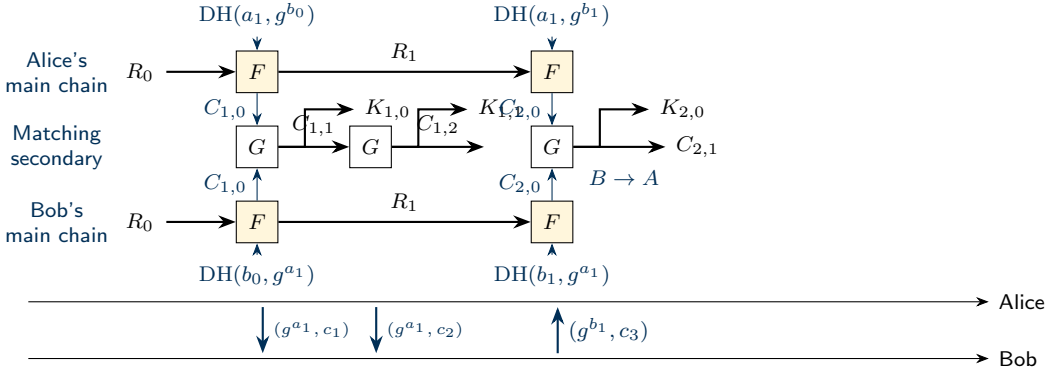
Second tick, same chain: $K_{1,1}$ encrypts m_2. No DH update.

Combining the Ratchets



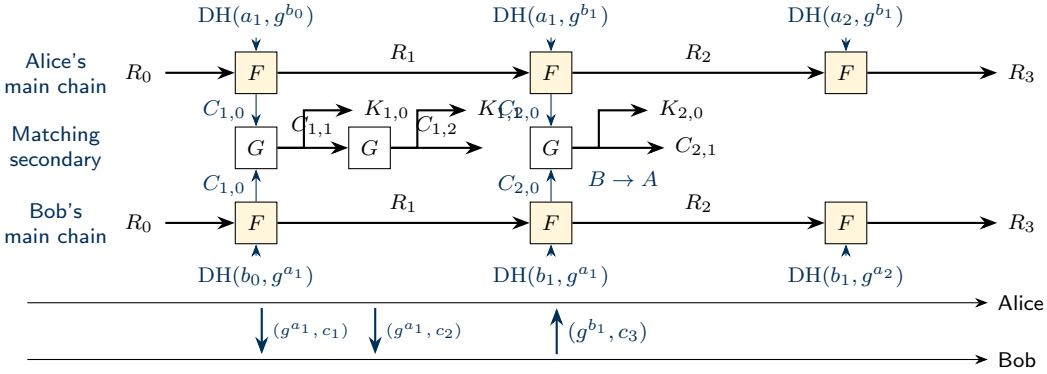
Bob samples b_1; the DH update starts a new Bob-to-Alice chain.

Combining the Ratchets



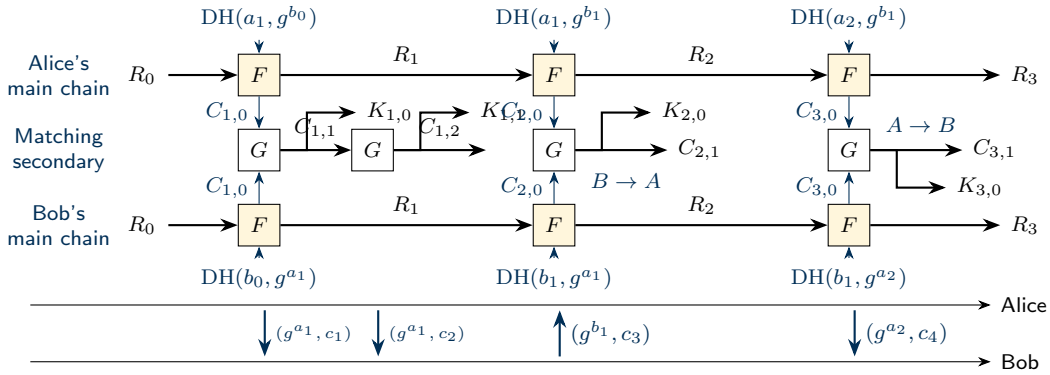
Tick that new chain: $K_{2,0}$ encrypts Bob's reply m_3.

Combining the Ratchets



Alice samples a_2; another DH update starts a new Alice-to-Bob chain.

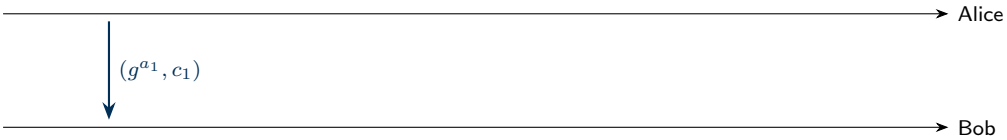
Combining the Ratchets



Alice sends twice on one secondary chain, then Bob replies on a fresh chain. The exchanges track the ticks above. This Double Ratchet idea is used by Signal and WhatsApp.

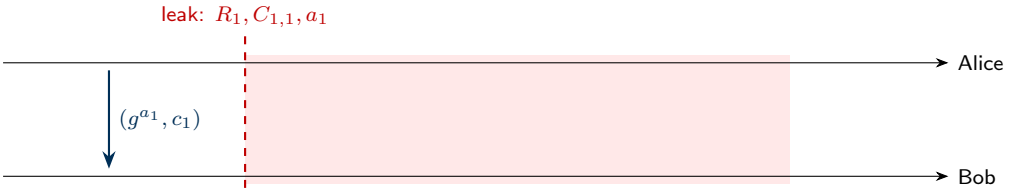
Leakage and Healing: Passive Attacker

One-time leak of Alice's state just after c_1; afterward the attacker only listens. Old states and used message keys were erased.



Leakage and Healing: Passive Attacker

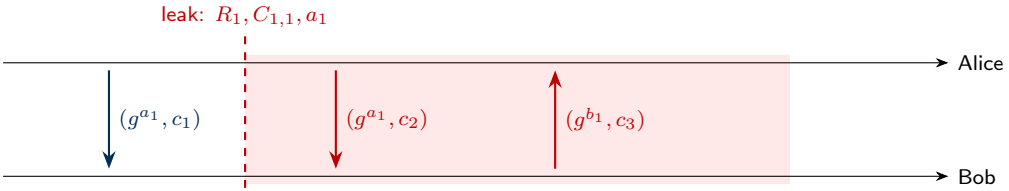
One-time leak of Alice's state just after c_1; afterward the attacker only listens. Old states and used message keys were erased.



- c_1: protected by erasure of its old message key and state.

Leakage and Healing: Passive Attacker

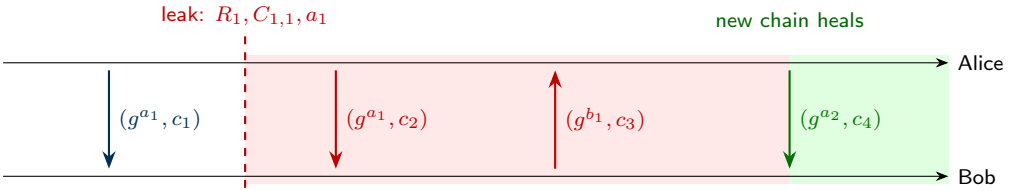
One-time leak of Alice's state just after c_1; afterward the attacker only listens. Old states and used message keys were erased.



- c_1: protected by erasure of its old message key and state.
- c_2: exposed by the stolen secondary-chain state $C_{1,1}$.
- c_3: also exposed; the attacker computes $X_2 = (g^{b_1})^{a_1}$.

Leakage and Healing: Passive Attacker

One-time leak of Alice's state just after c_1; afterward the attacker only listens. Old states and used message keys were erased.



- c_1: protected by erasure of its old message key and state.
- c_2: exposed by the stolen secondary-chain state $C_{1,1}$.
- c_3: also exposed; the attacker computes $X_2 = (g^{b_1})^{a_1}$.
- c_4 and its new chain: protected by fresh $X_3 = g^{a_2 b_1}$, unknown to the attacker. Previously exposed chains stay exposed.